

Guia do paper (para o Rafa)

Este documento não vai junto na entrega. Ele existe para você **entender e defender** cada parte do paper na conversa com a AdzHub. Lê uma vez antes da entrevista e você fala de tudo com segurança.

A tese em uma frase

O erro comum é tratar tudo como "tool". Eu proponho um harness **híbrido** onde cada uma das três camadas do domínio ganha um mecanismo diferente: a **memória** (supercérebro) é hidratada como **ambiente** antes do loop, as **APIs** (Meta, CRM) são **tools** num loop ReAct, e os **Apps** de metodologia são **skills** encapsuladas.

Se te perguntarem "resuma sua tese", é isso. O nome é **Harness Tri-Camada**.

Por que isso importa (a intuição)

O gestor de marketing não faz perguntas soltas. Toda pergunta dele pressupõe o **histórico da conta** ("o CPA subiu", "por que os leads dizem Google"). Se você trata a memória como só mais uma tool que o LLM chama quando lembra, dois problemas aparecem:

1. O LLM esquece de buscar contexto e responde no vácuo.
2. Ou busca demais, gastando passos e tokens.

A sacada é: **memória não é uma pergunta que o agente faz, é o chão onde ele pisa**. Então o harness recupera o contexto relevante e injeta ANTES do loop começar. O agente já nasce situado. Isso é o que a literatura chama de "contexto como ambiente" (RLM).

Ao mesmo tempo, os **Apps** (análise de criativos, mapa de solução) são metodologia pronta da AdzHub. Se o agente reimplementasse a metodologia no prompt, você teria duas versões divergindo. Então o harness **chama o app** e usa a saída. Isso é "skill".

E as **APIs** (Meta, CRM) são dado cru, sem estado. Essas sim são tools clássicas que o LLM decide chamar. O trabalho inteligente ("cruzar gasto do Meta com venda do CRM por utm_content") é do agente, não da tool.

A analogia do cérebro e do exoesqueleto (a sua, e está no paper)

- O **LLM** é um cérebro num pote: sabe muito, mas não vê o mundo nem age.
- O **harness** é o exoesqueleto: restringe o foco, dá músculos e ferramentas (tools, memória, dados) e força a saída num comportamento estrito.
- **Não muda o cérebro**. Não mexemos nos pesos do modelo (isso seria treino). Só embrulhamos software em volta e filtramos o que ele vê.

Serve para explicar em 20 segundos a diferença entre **modelo** e **harness**, que é a primeira coisa que o paper separa.

Passeio pelo paper, seção por seção

Abstract (Resumo)

O que diz: o problema do gestor, a tese (híbrido de 3 mecanismos), o que o protótipo ilustra e o recorte consciente (só-leitura). **Por que está aí:** um avaliador lê o abstract e decide se entendeu sua tese em 15 linhas. É o elevador. **Como defender:** "Ataco tarefas compostas do gestor (cruzar Meta x CRM, diagnosticar CPA, montar pauta) tratando cada camada do domínio com o mecanismo certo."

1. Introdução

O que diz: separa **modelo** (o LLM, prevê token) de **harness** (o runtime: loop, tools, limites, estado). Traz a analogia do exoesqueleto. Fecha com a frase de tese. **Por que está aí:** responde a pergunta 1 (tese), a 2 (o que deixa de ser chatbot) e a 3 (o que mudou na minha leitura depois de estudar). **Como defender:** "No começo eu achava que harness era só o loop de tool-calling. Estudando, entendi que a decisão de arquitetura não é 'qual loop', é **como cada recurso do domínio entra no runtime**: o que vira tool, o que vira memória, o que vira skill."

2. Preliminares

2.1 define o vocabulário (loop, tool, observação, memória, allowlist, max_steps, estado). **2.2** lê o domínio AdzHub em três camadas (supercérebro / Apps / APIs) e crava a observação central: essas camadas têm **naturezas diferentes**. **Por que está aí:** para o resto do paper usar as palavras com precisão. Um paper que não define os termos vira opinião. **Como defender:** "Antes de escolher a arquitetura eu preciso do vocabulário. E a chave é que as três camadas não são a mesma coisa embrulhada diferente."

3. Arquitetura (o coração)

3.1 declara a escolha (híbrido próprio, núcleo ReAct) e **justifica contra cada uma das 5 referências**, não só descreve a minha. Isso é o que a pergunta 4 pede. **3.2** responde a pergunta 5 (o que é tool, o que é memória, o que é app), item por item. **3.3** lista os trade-offs aceitos de propósito (pergunta 6). **Figura 1** desenha tudo: gestor → chat → harness (hidrata / loop / skills) → 3 camadas → ação. **Como defender cada rejeição:**

- **ReAct puro:** certo como esqueleto, mas sozinho vira "memória é tool" e o agente responde sem chão. Mantenho o loop, tiro a memória de dentro dele.
- **Sandbox/CodeAct:** o trabalho aqui não é rodar código, é cruzar tabelas. Sandbox adiciona risco e latência sem resolver o problema. Fica fora do MVP.
- **Permissões & skills:** adoto a metade de **skills** (os Apps são skills). Adio **permissões** porque o MVP é só-leitura.
- **Grafo de estados:** ótimo para fluxo fixo. O chat do gestor é aberto, uma tarefa por mensagem. O supercérebro é um grafo de **dados**, não de **controle**. Uso o grafo como memória, não como máquina de estados.
- **RLM (contexto como ambiente):** é a peça que cura a fraqueza do ReAct puro. É de onde vem a "hidratação".

4. Sistema (o que o protótipo ilustra)

O que diz: descreve o protótipo (dois motores: simulado e LLM), o trace visível, e um **turno concreto** (intent → hidratação → tools → observação → resposta) para o diagnóstico do Ômega 3. Tem a **Tabela 1** (cada peça: tool/memória/app, papel, onde vive) e explica os datasets e o campo de OpenRouter. **Por que está aí:** responde as perguntas 8 (o que o protótipo ilustra), 9 (datasets, o que é fake) e 10 (OpenRouter na UI). **Como defender:** "O protótipo não executa a infra do paper. Ele **simula** o resultado: as tools leem um mock cruzado, e o trace mostra o harness orquestrar. O avaliador vê a tese acontecendo."

5. Notas de execução

O que diz: PDF, demo estática, OpenRouter com key só no browser, dados mock determinísticos. **Por que está aí:** deixa claro o que é real e o que é fake, sem inventar métricas.

6. Limitações (a seção que dá credibilidade)

O que diz: onde a solução **quebra**, tarefa por tarefa do gestor (relatório, diagnóstico, pauta, criativos). E "com mais uma semana eu faria X, e deliberadamente NÃO faria Y". **Por que está aí:** responde a pergunta 7 (onde quebra) e a 11 (próxima semana). Gaps honestos valem pontos: mostram que você conhece os limites. **Como defender:** "O join Meta x CRM depende de UTM limpo; no mundo real UTM falta e a atribuição diverge. A hidratação rasa pode não achar o nó certo por sinônimo. O agente correlaciona, não prova causa."

Referências

OpenHands SDK, ReAct, CodeAct, RLM, Zep/Graphiti, Anthropic. São as fontes que sustentam cada decisão. Só citei o que de fato usei.

Onde cada uma das 11 perguntas do roteiro é respondida

#	Pergunta	Onde no paper
1	Qual a tese?	Abstract + §1 (frase de tese)
2	O que deixa de ser chatbot e vira agente?	§1 (modelo vs harness) + §3.2
3	O que você achava e o que mudou?	§1 (segundo parágrafo)
4	Qual abordagem escolheu e por quê vs as outras?	§3.1 (justifica contra as 5)
5	Como conversa com supercérebro/Apps/APIs? O que é tool/memória/app?	§3.2
6	Trade-offs aceitos de propósito	§3.3
7	Onde a solução quebra nas tarefas reais	§6 (por tarefa)
8	O que o protótipo ilustra vs só no paper	§4.1
9	Datasets, o que é fake, o que dá pra testar	§4.2 + Tabela 1
10	Como colar a OPENROUTER_API_KEY e trocar modelo	§4.3
11	Com mais uma semana, o que faria / não faria	§6 (final)

Glossário (para não travar em nenhum termo)

- **Harness:** o runtime em volta do LLM. O loop que executa tools, injeta observações, guarda estado e impõe limites. Não é o modelo.
- **Modelo (LLM):** prevê o próximo token. No máximo emite a intenção de chamar uma função.
- **Tool:** função com assinatura declarada que o modelo pede para chamar; devolve JSON (a observação) que volta ao contexto.
- **Observação:** o resultado de uma tool, reinjetado como nova evidência no loop.
- **Loop ReAct:** ciclo intenção → ação (tool) → observação → intenção, até responder ou bater um limite. (Reasoning + Acting.)

- **Memória como ambiente (RLM):** em vez de o LLM chamar a memória, o harness recupera o contexto relevante e injeta no prompt antes do loop. O agente já começa situado. RLM = Recursive Language Models (contexto vive fora da janela e o agente navega/fatia).
 - **Hidratação:** o ato de montar esse "pacote de contexto" (nós do grafo + eventos recentes da timeline) e injetar. No protótipo é uma fase visível no trace.
 - **Skill (App):** metodologia encapsulada que o harness invoca; não é reimplementada no prompt. Ex.: análise de criativos devolve ranking + recomendação (seguir/pausar/variá).
 - **Allowlist:** conjunto fechado de tools que podem ser chamadas. Guard-rail de harness.
 - **max_steps:** teto de passos por turno. Guard-rail de harness. No protótipo é 8.
 - **GraphRAG:** RAG (busca aumentada por recuperação) sobre um grafo de conhecimento, não sobre texto solto. É o formato do supercérebro.
 - **Supercérebro:** a memória da operação: pessoas, cliente, campanhas, decisões, ligadas em grafo, com linha do tempo (contexto temporal). Estilo Mem0 / Graphiti.
 - **CodeAct:** harness onde o agente escreve e executa código num sandbox. Rejeitado aqui.
 - **utm_content:** o parâmetro de URL que casa o anúncio do Meta (`ad_id`) com o lead no CRM. É a chave do "join" que revela o custo por venda real.
-

Perguntas que o avaliador pode fazer (e boas respostas)

"Por que não um grafo de estados, se o domínio já é um grafo?" Porque o supercérebro é um grafo de **dados**, não de **controle**. O chat é aberto: cada mensagem é uma tarefa diferente. Um grafo de controle fixo engessaria. Uso o grafo como memória.

"Por que não deixar tudo como tool e simplificar?" Porque memória e metodologia têm custo se viram tool. A memória vira adivinhação (o LLM esquece de buscar), e a metodologia vira reimplementação no prompt (cara e divergente). Cada natureza pede um mecanismo.

"Seu protótipo não executa a infra de verdade. Isso não enfraquece?" Não, porque o desafio pede para **simular/demonstrar o resultado**. O que prova a tese é o trace: o harness hidrata memória, chama tools de API e invoca skills, e a resposta é ancorada nos números do mock, não num palpite. Os 5 problemas plantados são descobertos, não estão escritos no dado.

"Onde isso quebra num cliente real?" UTM sujo quebra o join; atribuição multi-toque diverge entre Meta e CRM; a hidratação rasa (sem embeddings) pode errar o nó por sinônimo; e o agente correlaciona, não prova causa. Está tudo em §6.

"E segurança? O agente pode pausar um anúncio errado?" Não. Por decisão de arquitetura o MVP é **só-leitura**. Nenhuma tool escreve na conta. O agente propõe, o humano executa. Isso remove a classe inteira de risco de ação indevida.

"Com mais tempo, o que faria primeiro?" Recuperação de memória de verdade (grafo temporal com embeddings e decaimento, estilo Graphiti/Zep), para a hidratação parar de depender de match por substring.

Os 5 problemas plantados no dataset (o que a demo prova)

O mock foi construído para esconder 5 problemas de gestão que o agente descobre cruzando fontes. Nenhum está rotulado no dado.

1. **Criativo caro vs barato:** o vídeo de depoimento gasta R\$4.200 para 2 vendas (CPA R\$2.100), enquanto o "antes/depois" gasta R\$900 para 9 vendas (CPA R\$100). Só aparece cruzando Meta x CRM.
2. **Criativo saturado:** o hook_rate do depoimento cai de 32% para 18% e a frequência sobe de 1,8 para 4,8. O App recomenda pausar.
3. **Origem inconsistente:** 12 leads dizem "Google", mas só 4 têm UTM de Google; os outros vieram do Meta (um UGC de Whey). Decidir verba pela origem declarada erraria o alvo.
4. **Aprovação travada:** a campanha de Creatina está parada porque a peça diz "resultado garantido", que o mapa de solução lista como proibido de falar. Só se descobre lendo timeline + WhatsApp + mapa.
5. **Spend vs budget:** o Ômega 3 gastou R\$6.200 contra um teto de R\$5.000, puxado pelo mesmo depoimento. Uma ação (pausar o depoimento) resolve os problemas 1, 2 e 5 de uma vez.

O ponto que impressiona: **um único achado (pausar o depoimento) costura três sintomas.** Isso é o que separa um agente de um chatbot que responde uma pergunta por vez.